

Lambert Intercept Trajectory

AEE 462 – Orbital Mechanics
MATLAB Numerical Analysis

Problem 2

Determine the departure burn required for an interceptor spacecraft to reach a target spacecraft after a **45-minute time of flight**. The target's orbit is first propagated to the intercept time using the two-body equations of motion. A Lambert transfer is then solved between the interceptor's initial position and the target's position at the intercept time.

Solution

The MATLAB script begins by defining the initial position and velocity vectors of both the interceptor and target spacecraft, along with Earth's gravitational parameter and the required 45-minute transfer time.

The target spacecraft is propagated from its initial state to the desired intercept time using MATLAB's `ode45` solver and the two-body equations of motion. This determines the target's future position vector, $\mathbf{r}_2$, at the intercept time. The interceptor's initial position is represented by $\mathbf{r}_1$.

Using $\mathbf{r}_1$ and $\mathbf{r}_2$, the Lambert transfer geometry is constructed. The chord length between the two spacecraft positions is calculated as:

$$\mathbf{c} = |\mathbf{r}_2 - \mathbf{r}_1|$$

The semiperimeter is then:

$$\mathbf{s} = (\mathbf{r}_1 + \mathbf{r}_2 + \mathbf{c}) / 2$$

The minimum possible semimajor axis is determined from:

$$\mathbf{a_min} = \mathbf{s} / 2$$

The semimajor axis, $\mathbf{a}$, of the required transfer orbit is found numerically using MATLAB's `fzero` function to solve the Lambert time-of-flight equation for the specified 45-minute transfer.

Once the semimajor axis is known, the Lambert angles α and β are calculated. These values are then used to determine the semilatus rectum, p , and orbital eccentricity, e , of the transfer ellipse.

The transfer angle, Δv , between the initial and final position vectors is calculated using their dot product. The Lagrange coefficients f and g are then determined:

$$f = 1 - (r_2 / p)(1 - \cos \Delta v)$$

$$g = (r_1 r_2 / \sqrt{(\mu p)}) \sin \Delta v$$

The required interceptor departure velocity is then calculated from:

$$v_{\text{transfer}} = (r_2 - f r_1) / g$$

Finally, the required departure burn is found by subtracting the interceptor's initial velocity from the required transfer velocity:

$$\Delta v = v_{\text{transfer}} - v_{\text{initial}}$$

The magnitude of this vector gives the total departure delta-v required to place the interceptor onto the Lambert transfer trajectory.

Results

Time of Flight:

45 minutes (2700 seconds)

Departure Delta-v Vector (km/s):

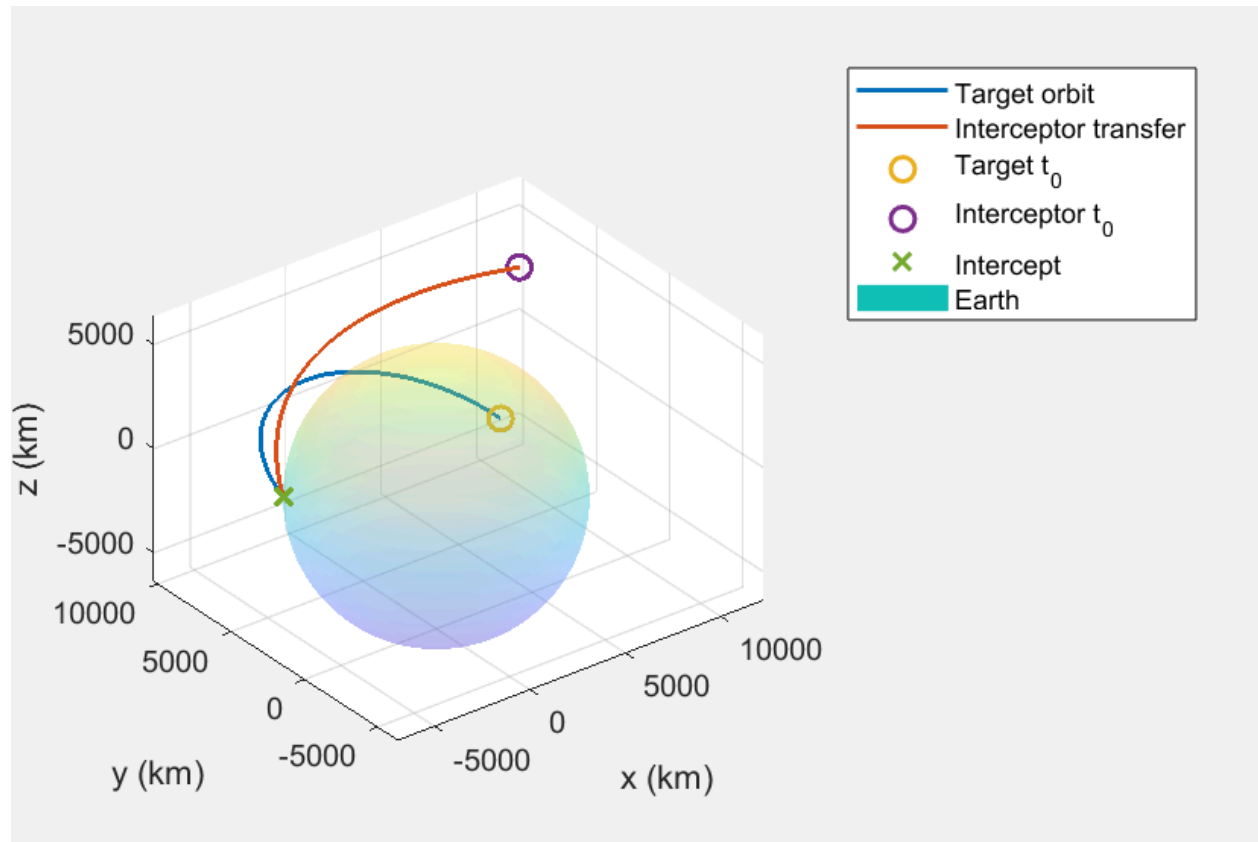
$$\Delta v = [-2.701111, -0.702232, 1.106929] \text{ km/s}$$

Departure Delta-v Magnitude:

$$|\Delta v| = 3.002403 \text{ km/s}$$

The required departure maneuver therefore has a total magnitude of approximately **3.00 km/s**.

After calculating the required departure velocity, the MATLAB script propagates the interceptor along the calculated transfer trajectory using `ode45`. A three-dimensional plot is generated showing the target orbit, interceptor transfer trajectory, initial spacecraft positions, intercept point, and Earth. This provides a visual representation of the Lambert intercept solution.



Lambert Intercept Transfer Trajectory Plot

Appendix – MATLAB Code

```
clear; clc;
%% AEE 462 HW 5 Pb 2
mu = 398600.4418; % km^3/s^2
dt = 45*60; % s
% Interceptor at t0
r_int0 = [12214.839; 10249.467; 2000]; % km
v_int0 = [-3.448; 0.924; 0]; % km/s
% Target at t0
r_tgt0 = [6045; 3490; 0]; % km
v_tgt0 = [-2.457; 6.618; 2.533]; % km/s
% Propagate target to intercept time
y0_tgt = [r_tgt0; v_tgt0];
opts = odeset('RelTol',1e-12,'AbsTol',1e-12);
[t_tgt, y_tgt] = ode45(@(t,y) twobody(t,y,mu), ...
    [0 dt], y0_tgt, opts);
r2_vec = y_tgt(end,1:3)'; % target pos at intercept
r1_vec = r_int0; % interceptor initial pos
% Lambert geometry
r1 = norm(r1_vec);
r2 = norm(r2_vec);
c_vec = r2_vec - r1_vec;
c = norm(c_vec);
s = (r1 + r2 + c)/2;
% Solve Lambert time equation for a
a_min = s/2;
fun_a = @(a) lambert(a, s, c, dt, mu);
a = fzero(fun_a, [a_min*(1+1e-8), 1e6]);
% alpha, beta, p, e
alpha = 2*asin(sqrt(s/(2*a)));
beta = 2*asin(sqrt((s-c)/(2*a)));
p = (4*a*(s-r1)*(s-r2)/c^2) * ...
    sin((alpha + beta)/2)^2;
e = sqrt(1 - p/a);
% Transfer departure velocity
dnu = acos(dot(r1_vec,r2_vec)/(r1*r2));
f = 1 - (r2/p)*(1 - cos(dnu));
g = (r1*r2/sqrt(mu*p))*sin(dnu);
v_tr0 = (r2_vec - f*r1_vec)/g;
% Departure delta-v
dV_vec = v_tr0 - v_int0;
dV = norm(dV_vec);
%% Results
```

```

fprintf('Results:\n');
fprintf('r2 (km):\n');
fprintf(['%.6f %.6f %.6f]\n\n', ...
    r2_vec(1), r2_vec(2), r2_vec(3));
fprintf('Lambert geometry:\n');
fprintf('r1 = %.6f km\n', r1);
fprintf('r2 = %.6f km\n', r2);
fprintf('c = %.6f km\n', c);
fprintf('s = %.6f km\n\n', s);
fprintf('Transfer ellipse:\n');
fprintf('a = %.6f km\n', a);
fprintf('alpha = %.6f rad = %.6f deg\n', ...
    alpha, rad2deg(alpha));
fprintf('beta = %.6f rad = %.6f deg\n', ...
    beta, rad2deg(beta));
fprintf('p = %.6f km\n', p);
fprintf('e = %.6f\n\n', e);
fprintf('v_tr(t0) (km/s):\n');
fprintf(['%.6f %.6f %.6f]\n\n', ...
    v_tr0(1), v_tr0(2), v_tr0(3));
fprintf('delta v vector (km/s):\n');
fprintf(['%.6f %.6f %.6f]\n\n', ...
    dV_vec(1), dV_vec(2), dV_vec(3));
fprintf('|delta-v| = %.6f km/s\n', dV);
%% Plot
y0_tr = [r_int0; v_tr0];
[t_tr, y_tr] = ode45(@(t,y) twobody(t,y,mu), ...
    [0 dt], y0_tr, opts);
figure;
plot3(y_tgt(:,1), y_tgt(:,2), y_tgt(:,3), ...
    'LineWidth', 1.5);
hold on;
plot3(y_tr(:,1), y_tr(:,2), y_tr(:,3), ...
    'LineWidth', 1.5);
plot3(r_tgt0(1), r_tgt0(2), r_tgt0(3), ...
    'o', 'MarkerSize', 8, 'LineWidth', 1.5);
plot3(r_int0(1), r_int0(2), r_int0(3), ...
    'o', 'MarkerSize', 8, 'LineWidth', 1.5);
plot3(r2_vec(1), r2_vec(2), r2_vec(3), ...
    'x', 'MarkerSize', 8, 'LineWidth', 1.5);
[xe,ye,ze] = sphere(60);
Re = 6378;
surf(Re*xe, Re*ye, Re*ze, ...
    'FaceAlpha', 0.2, 'EdgeColor', 'none');

```

```

axis equal;
grid on;
xlabel('x (km)');
ylabel('y (km)');
zlabel('z (km)');
legend('Target orbit', ...
       'Interceptor transfer', ...
       'Target t_0', ...
       'Interceptor t_0', ...
       'Intercept', ...
       'Earth');
%% Functions
function dydt = twobody(~, y, mu)
    r = y(1:3);
    v = y(4:6);
    rnorm = norm(r);
    a = -mu/rnorm^3 * r;
    dydt = [v; a];
end
function F = lambert(a, s, c, dt, mu)
    alpha = 2*asin(sqrt(s/(2*a)));
    beta = 2*asin(sqrt((s-c)/(2*a)));
    tof = sqrt(a^3/mu) * ...
        (alpha - beta - (sin(alpha) - sin(beta)));
    F = tof - dt;
end

```